Frequency-based Text Representations

Data Science with Applications in Natural Language Processing

Goals

By the end, we will be able to:

- **build** a vocabulary, one-hot and bag-of-words vectors
- **use** CountVectorizer and TfidfVectorizer
- **compute** TF-IDF by hand, exactly as sklearn does
- **tune** vocabulary, n-grams and stopwords, knowing the pitfalls
- **find** local language patterns in Greek proverbs

Outline

1. From text to vocabulary
2. Bag-of-words and sparse matrices
3. TF-IDF
4. Hyperparameters and pitfalls
5. Application: dialect in Greek proverbs

A question

«Ου Θεός κι ου γείτονας»

“Neither God nor the neighbour”, with **ου** instead of standard **ο**

Which region is this proverb from?

- We do not need to understand the **meaning**
- It is enough to **count** forms: «ου» instead of «ο»

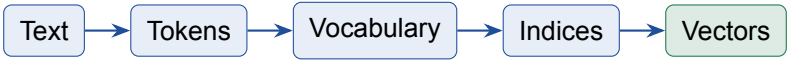
The idea

The meaning of a word lies in its use

- Which words appear, and how often
- $\Rightarrow$ a vector of numbers per text
- $\Rightarrow$ classification, clustering, search

Wittgenstein, as quoted by Firth (1957).

The pipeline



- Every step is a **choice** that changes the result

Tokenization

```
"the cat sat on the mat".split()
# ['the', 'cat', 'sat', 'on', 'the', 'mat']
```

- Token: a word, a character or a **subword**
- Often combined with cleaning:
  - lowercasing, punctuation, stopwords
  - stemming, lemmatization

More on this topic: data processing (text cleaning).

Vocabulary: unique words

```
tokens = "the cat sat on the mat".split()
V = sorted(set(tokens))
# ['cat', 'mat', 'on', 'sat', 'the']
w2i = {w: i for i, w in enumerate(V)}
# {'cat': 0, 'mat': 1, 'on': 2, 'sat': 3, 'the': 4}
```

- Each vocabulary word = one **dimension**
- set: no duplicates. sorted: a stable order

Pitfall: a vocabulary with duplicates

```
V = "the cat sat on the mat".split()     # not unique!
{w: i for i, w in enumerate(V)}
# {'the': 4, 'cat': 1, 'sat': 2, 'on': 3, 'mat': 5}
```

- 6 dimensions for 5 words
- “the” maps to position 4, but V.index("the") returns 0
- No error, just **inconsistent** vectors

One-hot

```
def one_hot(V, word):
    vec = [0] * len(V)
    vec[w2i[word]] = 1
    return vec

one_hot(V, "the")      # [0, 0, 0, 0, 1]
one_hot(V, "mat")      # [0, 1, 0, 0, 0]
```

- Length = vocabulary size, a single 1
- All words are **equally distant**: cat, mat, the

From words to texts

	cat	mat	on	sat	the
the	0	0	0	0	1
mat	0	1	0	0	0
the	0	0	0	0	1
cat	1	0	0	0	0
sat	0	0	0	1	0
sum	**1**	**1**	**0**	**1**	**2**
binary	**1**	**1**	**0**	**1**	**1**

- Sum of one-hots = **counts**. Capped at 1 = **presence**

In code

```
import numpy as np
def bag_of_words(tokens, V, binary=True):
    bow = np.sum([one_hot(V, w) for w in tokens], axis=0)
    return np.minimum(bow, 1) if binary else bow

bag_of_words(["the", "mat", "the", "cat", "sat"], V)
# array([1, 1, 0, 1, 1])
```

The book has np.clip(0, bow, 1). It works by accident: the right order is np.clip(bow, 0, 1).

In practice: CountVectorizer

```
from sklearn.feature_extraction.text import CountVectorizer
corpus = ["the cat sat on the mat", "the cat sat", "the mat"]
cv = CountVectorizer()
X = cv.fit_transform(corpus)
cv.get_feature_names_out()   # ['cat' 'mat' 'on' 'sat' 'the']
X.toarray()
```

	cat	mat	on	sat	the
doc 0	1	1	1	1	2
doc 1	1	0	0	1	1
doc 2	0	1	0	0	1

CountVectorizer(binary=True) for presence.

Pitfall: one-letter words

```
cv = CountVectorizer()
cv.fit(["I have a cat and a dog"])
cv.get_feature_names_out()
# ['and' 'cat' 'dog' 'have']
```

- Where did “I” and “a” go?
- The default token_pattern keeps tokens of **2+ characters**
- Worse in Greek: ο, η, κ’, θ’ are very common

```
CountVectorizer(token_pattern=r"(?u)\b\w+\b")   # keeps 'i', 'a'
```

fit and transform

```
cv = CountVectorizer().fit(corpus)          # learns vocabulary
cv.transform(["the dog sat on the sofa"]).toarray()
# [[0 0 1 1 2]]       cat mat on sat the
```

- fit: learns the vocabulary
- transform: counts **only** vocabulary words
- dog, sofa: **unknown**, silently dropped

Sparse matrices

- Vocabulary of 10,000 words, a text with 100 unique ones
- $\Rightarrow$ 9,900 zeros per row (99%)
- Solution: store **only** the non-zero values and their positions

row	column	value
0	4 (the)	2
0	0 (cat)	1
…		

CSR

```
X
# <Compressed Sparse Row sparse matrix of dtype 'int64'
#   with 10 stored elements and shape (3, 5)>
```

- 15 cells, 10 stored, 5 zeros not stored
- X.toarray(): a dense matrix, **only** for small data
- For the proverbs: $23 \times 102{,}219$, only 9.8% non-zero

Which word matters?

the *term data science and* ***the*** *real world*

- “the” appears twice, every other word once
- By raw counts, “the” looks like the most important word
- But “the” is **everywhere**: it distinguishes no text

Two components

TF: term frequency

- how frequent the word is **in this** text
- $TF(\text{the}) = 2/8$

IDF: inverse document frequency

- how **rare** it is in the corpus
- common word $\Rightarrow$ small weight

TF-IDF = TF $\times$ IDF

IDF

$$IDF(w) = \log \frac{|D|}{|\{d \in D : w \in d\}|}$$

- $|D|$: number of texts. Denominator: texts that contain w
- In **all** texts: $\log 1 = 0$
- In 1 of 3: $\log 3 = 1.10$
- The logarithm compresses large differences

sklearn's IDF is slightly different

$$IDF(w) = \ln \frac{1 + |D|}{1 + df(w)} + 1$$

	cat	mat	on	sat	the
df (in 3 texts)	2	2	1	2	3
$\log(\|D\|/df)$	0.41	0.41	1.10	0.41	**0.00**
sklearn	1.29	1.29	1.69	1.29	**1.00**

- The $+1$: a word that is everywhere is **not** zeroed out

  smooth_idf=True: as if one extra text contained every word.

TfidfVectorizer

```
from sklearn.feature_extraction.text import TfidfVectorizer
tv = TfidfVectorizer()
X = tv.fit_transform(corpus)
X.toarray().round(2)
```

	cat	mat	on	sat	the
the cat sat on the mat	0.37	0.37	**0.49**	0.37	**0.58**
the cat sat	0.62	0	0	0.62	0.48
the mat	0	0.79	0	0	0.61

“the” is still the largest in doc 0. Why?

By hand: “the cat sat on the mat”

	cat	mat	on	sat	the
TF (count)	1	1	1	1	2
IDF	1.29	1.29	1.69	1.29	1.00
TF $\times$ IDF	1.29	1.29	1.69	1.29	2.00
$\div \|\| \cdot \|\|_2 = 3.44$	0.37	0.37	0.49	0.37	**0.58**

- TF in sklearn = a **raw count**, not a proportion
- L2 normalisation: every row has length 1
- “the”: TF 2 $\times$ IDF 1 $>$ TF 1 $\times$ IDF 1.69

Why normalise?

- A long text has large counts everywhere
- Without normalisation, **length** dominates **content**
- With L2, we compare **directions**: cosine similarity

norm='l2' (default), 'l1' or None.

What we tune

Parameter	What it does
max_features=5000	keeps the 5000 **most frequent** terms
min_df=2	drops terms in fewer than 2 texts
max_df=0.5	drops terms in more than 50% of texts
ngram_range=(1, 2)	words and word pairs
stop_words="english"	removes a list of common words
analyzer="char_wb"	**character** n-grams inside words

max_features: “most frequent”, not “most important”.

n-grams: a little word order

```
tv = TfidfVectorizer(ngram_range=(1, 2)).fit(["the cat sat"])
tv.get_feature_names_out()
# ['cat' 'cat sat' 'sat' 'the' 'the cat']
```

- “not good” vs “good”: the bigram “not good” makes the difference
- Cost: the vocabulary grows a lot

Pitfall: stopwords and bigrams

```
tv = TfidfVectorizer(stop_words="english", ngram_range=(1, 2))
tv.fit(corpus).get_feature_names_out()
# ['cat' 'cat sat' 'mat' 'sat' 'sat mat']
```

- “sat mat”? The text says “sat *on the* mat”
- Stopwords are removed **before** the bigrams are formed
- “on” and “the” are gone, and with them the meaning

Also, there is no stop_words="greek": you need your own list.

Remember word forms?

*“**data** science”* vs *“**datasets** and language”*

Representation	cosine similarity
words	**0.00**
character n-grams (3–5)	**0.17**

- Shared: da, dat, data, … (6 n-grams)
- Character n-grams capture **stems** and **endings**
- Even stronger in Greek: «δεδομένων» vs «δεδομένα» share 15

char_wb

```
cv = CountVectorizer(analyzer="char_wb", ngram_range=(3, 3))
cv.fit(["datasets"]).get_feature_names_out()
# [' da' 'ase' 'ata' 'dat' 'ets' 'set' 'tas' 'ts ']
```

- wb = word boundaries: n-grams **inside** each word
- Spaces at the edges mark the **start** and **end** of a word
- Ideal for endings, phonetic variants, spelling

Limits of frequency representations

- They do **not** know that “car” $\approx$ “automobile”
- Word order only locally (n-grams)
- Very high dimensionality

\+ Fast, **interpretable**, a strong baseline

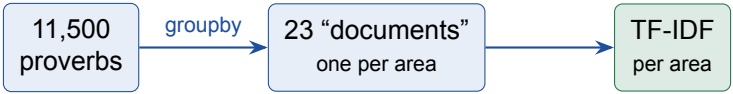
More on this topic: vector representations (embeddings).

The data

```
url = ("https://raw.githubusercontent.com/greek-proverb-atlas/"
       "proverbs.gr/main/data/balanced_corpus.csv")
data = pd.read_csv(url)
data.shape                    # (11500, 8)
data.area.value_counts()      # 23 areas x 500 proverbs
```

- Greek proverbs with the **area where recorded** and the collector
- **Balanced**: 500 proverbs per area

An idea: each area as one “document”



- IDF now counts **how many areas** use a pattern
- A pattern of one area $\Rightarrow$ high value
- Common to all $\Rightarrow$ low

This works because the areas have the same number of texts. Why?

One text per area

```
texts = data.groupby("area")["text"].apply("\n".join)
texts.shape                    # (23,)
```

```
tv = TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5),
                     max_df=0.5, min_df=1)
X = tv.fit_transform(texts)
X.shape                        # (23, 102219)
tfidf = pd.DataFrame(X.toarray(), index=texts.index,
                     columns=tv.get_feature_names_out())
```

max_df=0.5: drops patterns found in more than 11 areas.

The most characteristic n-grams

```
top = {area: tfidf.loc[area].nlargest(10) for area in tfidf.index}
```

Area	top n-grams	
Cyprus	τζ', τζαι	«τζαι» = and
Skyros	τσαί, ρζι	«τσαί» = and
Euboea	τσαί	
Lesbos	τσι	«τσι» = and
Aetolia	ου , είνι, μι	ο → ου, ε → ι
Asia Minor	τουν, είνιν	
Pontus	σο , ατ'	

nlargest returns **values**, not just names.

Tsitakism

*Άθθρωπος του σσοινιού **τζαι** του παλλουτζιού*
Cyprus

*Λίγα **τσι** παστρικά*
Lesbos: “few **and** clean”

*Τετράδη **τσαί** Παραστσευή τα νύχια σου μη κόψεις*
Skyros: “on Wednesday **and** Friday do not cut your nails”

- κ before ε/ι is pronounced ts or tz
- «τζαι» appears **only** in Cyprus (54 proverbs)

Northern dialects

*Αλίμουνου στουν αρφανό κι' ας **είνι** κι μι γένεια*
Aetolia: "woe to the orphan, even if he **is** bearded"

- Unstressed ο $\rightarrow$ ου, ε $\rightarrow$ ι: «είναι» $\rightarrow$ «είνι»
- «είνι» in 7 areas: Aetolia, Lesbos, Asia Minor, Epirus…
- Shared by many areas $\Rightarrow$ **lower IDF**
- TF-IDF highlights what is **unique**, not what a group of areas **shares**

Dialect or collector?

Area	top n-grams	what is it?
Karpathos	(δ), (γ), (ν)	**transcription** convention: «ά(δ)ικον»
Achaea	ι! , ει!	**exclamation marks**: 182 of 500
Thesprotia	(το)	«Καλό (το)…»: collection **index**

- Achaea: **95%** of proverbs from one collector
- Skyros: **99.6%** from one collector
- The model cannot separate area from **collector**: confounding

The role of max_df

```
tfidf.loc["Κύπρος"].idxmax()      # Cyprus
```

	Cyprus	Skyros	Crete
max_df=0.5	τζ'	σαί	ντου
max_df=1.0	το	τσ	το

- With 23 “documents”, IDF has few possible values
- το (“the”) is everywhere, but its huge TF wins
- max_df does the job IDF cannot do here

Accents: τσαί or τσαι?

```
tv = TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5),
                     max_df=0.5, strip_accents="unicode")
```

- Without it: τσαί and τσαι are **different** n-grams
- With strip_accents: vocabulary drops from 102,219 to 71,918
- Cyprus: τζαι, Skyros: τσαι

In dialect, stress can be a **signal**. Here it is probably spelling noise.

A threshold for presentation

- The book keeps TF-IDF values > 0.15 to show the “strong” patterns
- **8** of the 23 areas remain
- The other 15 are **not** without dialect
  - their patterns are simply shared by many areas
- Always report the threshold, and what it **hides**

More on this topic: visualisation (heatmaps).

The level of analysis is a choice

“Document” =	IDF counts…
one proverb	in how many proverbs
one area	in how many areas
one collector	in how many collectors
one decade	in how many decades

- The same method answers **different** questions

Summary

- Vocabulary of **unique** terms $\rightarrow$ one-hot $\rightarrow$ bag-of-words
- Sparse matrices: store only the non-zeros
- TF-IDF: frequent **here**, rare **elsewhere**
- sklearn: TF = count, IDF with $+1$, L2 normalisation
- Character n-grams: inflection, spelling, dialect
- “Characteristic” patterns may belong to **the collector**

Exercise 1: by hand

Three texts: "a b a", "a c", "b c d"

1. Write the bag-of-words matrix (counts)
2. Compute sklearn's IDF for each term
3. Which term has the highest TF-IDF in the first text?

Careful: one-letter words need a token_pattern.

Exercise 1: solution

	a	b	c	d
"a b a"	2	1	0	0
"a c"	1	0	1	0
"b c d"	0	1	1	1
df	2	2	2	1
IDF $= \ln \frac{4}{1+df} + 1$	1.29	1.29	1.29	1.69

- First text: a $= 2 \times 1.29$, b $= 1.29 \Rightarrow$ **a** (0.89 vs 0.45 after L2)

```
TfidfVectorizer(token_pattern=r"(?u)\b\w+\b").fit_transform(docs)
```

Exercise 2: the proverbs

1. Repeat the analysis with “document” = **collector**. What changes?
2. Compare analyzer="word" with "char_wb": which shows dialect better?
3. Remove ! and parentheses before TF-IDF. Which areas change?

For 3: data.text.str.replace(r"[!()]", "", regex=True): the brackets mean “any one of the characters !, (,).

Exercise 2: solution

1. **Collector**: 272 “documents”. For Ch. Koryllos (95% of Achaea) the top n-gram is again ι! : the exclamation mark follows the **person**, not the place
2. **Words**: τζ, τζαι (Cyprus); τσαί (Skyros); τσι, είνι (Lesbos); ου, είνι, μι (Aetolia). Short function words suffice; character n-grams also catch **endings**
3. **Without** !(): Achaea, Thesprotia, Karpathos, Rhodes change
   - Karpathos: from (δ) to τσαι, the tsitakism the transcription “hid”
   - Rhodes: from (μ) to λωλλ

Cyprus does not change: τζ' is linguistic, not punctuation.

For discussion

1. Why does joining texts per area require **balanced** data?
2. An area has a single collector. What can we say about its language?
3. When would you keep accents and punctuation?
4. When would you prefer bag-of-words to a modern language model?